НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
МОСКОВСКИЙ ЭНЕРГЕТИЧЕСКИЙ ИНСТИТУТ НИУ «МЭИ»
(ФГБОУ ВО «НИУ МЭИ»)
Институт радиотехники и электроники им. В.А. Котельникова
Кафедра Основ радиотехники

Реферат

“Алгоритм разложения сигнала на эмпирические моды”

Группа:
ЭР-16м-25
Студент:
Колесник Н.А.
Преподаватель:
Копылова Е.В.

Москва, 2025



Рисунок 1 – Алгоритм EMD

3 Пример применения модовой декомпозиции

Для примера был создан сигнал, состоящий из трех гармонических составляющих и добавлен шум.

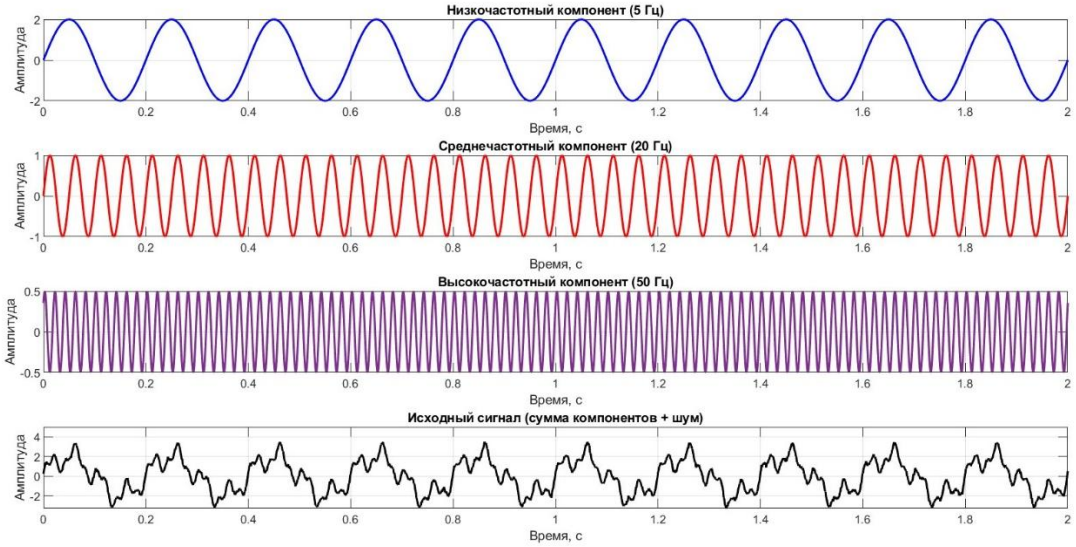


Рисунок 2 – Исходный сигнал из 3х компонент и шума

После разложения сигнала были получены 6 внутренних колебаний.

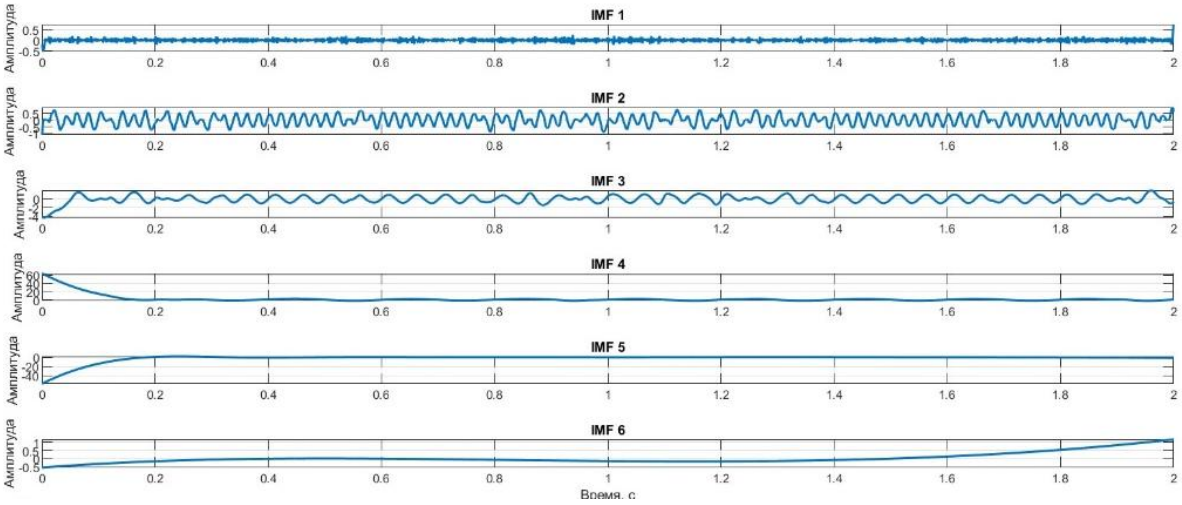


Рисунок 3 – Разложение на 6 внутренних колебаний

Для сравнения с исходным сигналом были просуммированы компоненты и была вычислена ошибка восстановления сигнала.

Ошибка восстановления сигнала мала.

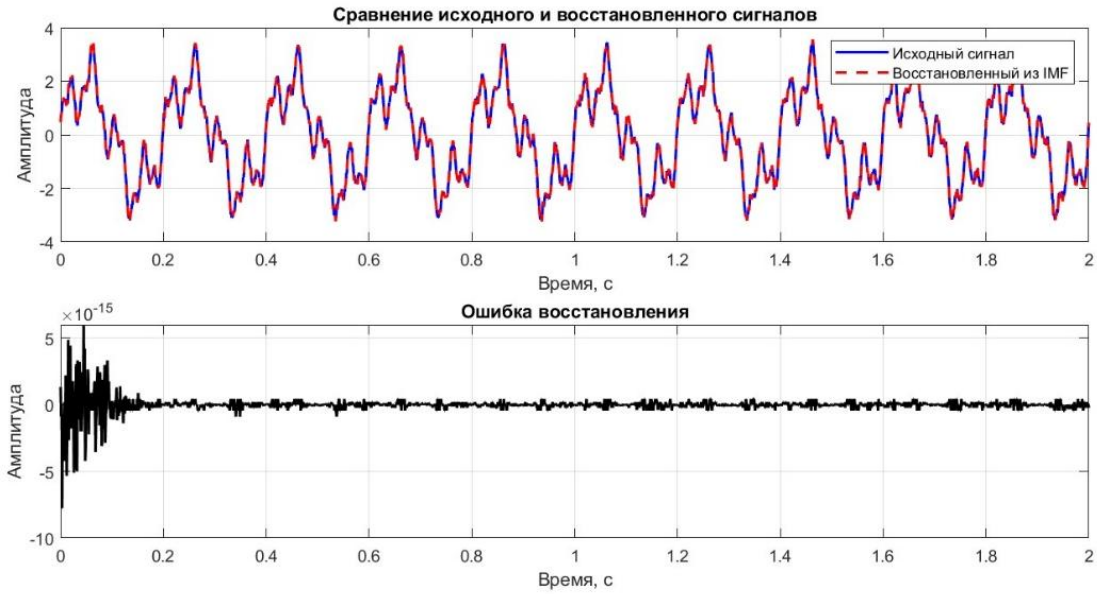


Рисунок 4 – Сравнение исходного сигнала и восстановленного

EMD может применяться в обработке биомедицинских сигналов для выделения физиологических компонентов и подавления артефактов, а также в геофизике для анализа сейсмических данных и колебаний океана. Метод также используется в радиолокации для разделения перекрывающихся эхо-сигналов, в вибродиагностике оборудования для детектирования дефектов и циклических компонент нестационарных процессов.

ПРИЛОЖЕНИЕ А

Листинг А.1 – Визуализация алгоритма

```
clear all; close all; clc;

%% Создание тестового сигнала
t = 0:1/fs:2;           % Временной вектор

% Компоненты сигнала
f1 = 5;                 % Низкочастотный компонент (5 Гц)
f2 = 20;                % Среднечастотный компонент (20 Гц)
f3 = 50;                % Высокочастотный компонент (50 Гц)

% Создание компонентов
comp1 = 2 * sin(2*pi*f1*t);             % Низкочастотный
comp2 = 1 * sin(2*pi*f2*t);             % Среднечастотный
comp3 = 0.5 * sin(2*pi*f3*t + pi/4);    % Высокочастотный

% Шум
noise = 0.1 * randn(size(t));

% Исходный сигнал = сумма компонентов + шум
signal = comp1 + comp2 + comp3 + noise;

%% Визуализация исходного сигнала и его компонентов
figure('Position', [100, 100, 1200, 800]);
subplot(4,1,1);
plot(t, comp1, 'b', 'LineWidth', 1.5);
title('Низкочастотный компонент (5 Гц)');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

subplot(4,1,2);
plot(t, comp2, 'r', 'LineWidth', 1.5);
title('Среднечастотный компонент (20 Гц)');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

subplot(4,1,3);
plot(t, comp3, 'g', 'LineWidth', 1.5);
title('Высокочастотный компонент (50 Гц)');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

subplot(4,1,4);
plot(t, signal, 'k', 'LineWidth', 1.5);
title('Исходный сигнал (сумма компонентов + шум)');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

%% Реализация EMD алгоритма
function imfs = emd(signal, max_imfs, max_iterations)
    % EMD - Эмпирическая модовая декомпозиция
    % signal - входной сигнал
    % max_imfs - максимальное количество IMF для извлечения
    % max_iterations - максимальное количество итераций для каждой IMF

    if nargin < 2
        max_imfs = 10;
    end
```

```
if nargin < 3
    max_iterations = 500;
end

residue = signal(:);  % Остаток
imfs = [];            % Матрица для IMF

for imf_idx = 1:max_imfs
    h = residue;  % Начальное приближение для текущей IMF

    for iter = 1:max_iterations
        % Находим локальные максимумы
        [max_peaks, max_locs] = findpeaks(h);

        % Находим локальные минимумы
        [min_peaks, min_locs] = findpeaks(-h);
        min_peaks = -min_peaks;

        % Проверяем, достаточно ли экстремумов
        if length(max_peaks) < 3 || length(min_peaks) < 3
            break;
        end

        % Интерполяция верхней и нижней огибающих
        t = 1:length(h);

        % Интерполяция кубическими сплайнами
        upper_env = interp1(max_locs, max_peaks, t, 'spline');
        lower_env = interp1(min_locs, min_peaks, t, 'spline');

        % Проверка на NaN
        if any(isnan(upper_env)) || any(isnan(lower_env))
            % Используем линейную интерполяцию
            upper_env = interp1(max_locs, max_peaks, t, 'linear', 'extrap');
            lower_env = interp1(min_locs, min_peaks, t, 'linear', 'extrap');
        end

        % Средняя огибающая
        mean_env = (upper_env + lower_env) / 2;

        % Вычитание средней огибающей
        h_new = h - mean_env';

        % Критерий остановки (стандартное отклонение)
        sd = sum((h - h_new).^2) / sum(h.^2);

        % Обновление
        h = h_new;

        % Условие остановки
        if sd < 0.3
            break;
        end
    end

    % Сохраняем полученную IMF
    imfs = [imfs; h'];

    % Обновляем остаток
    residue = residue - h;
```

```
        % Критерий остановки для EMD
        if length(findpeaks(residue)) < 3 || length(findpeaks(-residue)) < 3
            % Сохраняем последний остаток как последнюю IMF
            imfs = [imfs; residue'];
            break;
        end
    end
end

%% Применение EMD к сигналу
fprintf('Применение EMD к сигналу...\n');
imfs = emd(signal, 10, 500);

%% Визуализация результатов EMD
figure('Position', [100, 100, 1200, 800]);

% Отображаем все IMF
num_imfs = size(imfs, 1);
for i = 1:num_imfs
    subplot(num_imfs + 1, 1, i);
    plot(t, imfs(i,:), 'LineWidth', 1.5);

    if i < num_imfs
        title(sprintf('IMF %d', i));
    else
        title(sprintf('Остаток (Residue)'));
    end

    if i == num_imfs
        xlabel('Время, с');
    end
    ylabel('Амплитуда');
    grid on;
end

% Исходный сигнал для сравнения
subplot(num_imfs + 1, 1, num_imfs + 1);
plot(t, signal, 'k', 'LineWidth', 1.5);
title('Исходный сигнал для сравнения');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

%% Восстановление сигнала из IMF
reconstructed_signal = sum(imfs, 1);

% Вычисление ошибки восстановления
reconstruction_error = signal - reconstructed_signal;
mse = mean(reconstruction_error.^2);
fprintf('Среднеквадратичная ошибка восстановления: %e\n', mse);

%% Визуализация восстановления
figure('Position', [100, 100, 1000, 600]);

subplot(2,1,1);
plot(t, signal, 'b', 'LineWidth', 1.5);
hold on;
plot(t, reconstructed_signal, 'r--', 'LineWidth', 1.5);
hold off;
title('Сравнение исходного и восстановленного сигналов');
xlabel('Время, с'); ylabel('Амплитуда');
legend('Исходный сигнал', 'Восстановленный из IMF');
```

```
grid on;

subplot(2,1,2);
plot(t, reconstruction_error, 'k', 'LineWidth', 1.5);
title('Ошибка восстановления');
xlabel('Время, с'); ylabel('Амплитуда');
grid on;

fprintf('Количество найденных IMF: %d\n', num_imfs);

for i = 1:num_imfs
    fprintf('\nIMF %d:\n', i);
    fprintf('  Среднее значение: %.4f\n', mean(imfs(i,:)));
    fprintf('  Стандартное отклонение: %.4f\n', std(imfs(i,:)));
    fprintf('  Энергия: %.4f\n', sum(imfs(i,:).^2));
end
```